

Helpflow - ITIL Desk

Contents

Introduction.....	2
Design Patterns.....	2
Full Stack Overview.....	3
Frontend	3
Backend.....	3
Database	3
System architecture	3
Directory structure	3
Presentation Layer (Frontend)	4
Application Layer (Frontend services + Backend API logic)	4
Data Layer (Database + Access)	4
ITIL	4
Diagrams	5
DB-Diagram	5
Sequence-diagram (Create Ticket)	6
Component-diagram	6
User Manuals.....	7
For users.....	7
For supporters.....	9
Lighthouse test	13
Dashboard	13
Tickets	13
Articles	13
SLA-management.....	13
Contact.....	13
Retrospective	14
Conclusion	14
Installation / API / Dependency setup	15

Introduction

This project was created to learn about ITIL and GUI programming. I have tried to create an intuitive design. That is easy to understand and navigate. The project was created as a school project during my second school term.

I have created a connection between the back- and front-end system with API calls that use CRUD and SELECT statements from the database.

The project itself is a service help desk that will help with communication and problem solving of technical issues that the user may be having. This will be resolved through a ticket system, where a user can describe their issue in detail. After the ticket is created, a timeline will be set according to the category of the problem. There will be possibility for communication between the user and the supporter through a comment system.

Design Patterns

The project is not built around one single design pattern but rather utilizing a few different.

- Component-based architecture

Each page is a standalone component (like tickets, sidebar, footer etc.). This allows for reusability of each component and a more organized directory structure.

- Service layer pattern

API calls are wrapped in services.

- Reactive state pattern

I am using Angular signals and computed values for derived UI state. This is similar to MVVM. Examples of this can be seen in the tickets.ts and sla-management.ts.

- Router-driven lazy loading

Pages are loaded by routes in app.routes.ts instead of all at once.

- Singleton

I am using a singleton state service pattern for the role switcher.

Overall, this design would be an Angular component-service architecture with reactive state handling.

The benefit of this design is that it has a clear separation between the UI, API logic and database operations and is easier to maintain and develop due to the separation of each component.

Full Stack Overview

Frontend

- Angular 21
- TypeScript 5.9
- Angular Material + CDK
- Component-scoped CSS (Angular styles)
- Vitest (via *ng test*)

Backend

- Express
- MySQL 8+

Database

- MySQL 8+ (Using helpflowdb schema)

System architecture

This project follows a layered, feature-oriented full-stack architecture built around Angular on the frontend, Express.js on the backend, and MySQL as the persistence layer.

Directory structure

- **{Helpflow-backend}** Node/Express API layer for tickets, articles and SLA endpoints
 - **{db}** Database connection setup
 - **{routes}** REST route handlers
 - **{node_modules}** Backend dependencies
- **{Helpflow-frontend}** Angular application (UI, routing, pages, components, services)
 - **{src/app}** Root and Core app code folder
 - **{layout/main-layout}** Main sidebar, content + footer wrapper
 - **{pages}** Route-level feature pages - containing all page views
 - **{services}** API/data and app-wide services
 - **{shared}** Reusable shared components like sidebar
 - **{public}** Static assets, SQL files and docs
 - **{.angular}** Angular CLI cache/build metadata
 - **{node_modules}** Frontend dependencies

Presentation Layer (Frontend)

The presentation layer is implemented as a Single Page Application (SPA) using Angular standalone components.

Its responsibility is to render pages and reusable UI/layout components, handle user interaction, and present data to the user.

This layer does not access the database or backend logic directly. Instead, it delegates communication to frontend service classes, which keeps templates and components focused on UI behavior.

It also controls view-level role behavior (User vs Supporter) through shared role state, conditional rendering, and route-level navigation rules.

Application Layer (Frontend services + Backend API logic)

The application layer contains orchestration and business behavior.

On the frontend, service classes encapsulate HTTP communication and expose typed methods to components. Components consume these services and transform data into view models for display.

On the backend, route/API handlers receive requests, validate inputs, apply business rules, coordinate multi-step operations and return standardized API responses.

This layer should also enforce authorization rules server-side to ensure restricted operations are protected beyond frontend visibility logic.

Data Layer (Database + Access)

The data layer is responsible for persistent storage and retrieval of domain entities such as tickets, SLAs, categories, articles, supporters, statuses, priorities, and comments.

It enforces relational integrity through primary keys and foreign keys, supports CRUD operations, and serves as the source of truth for application data.

ITIL

Incident & request management

The ticket system is the core. Users create tickets with priority, category, and description.

Tickets flow through statuses (Open → In Progress → Resolved → Closed) with escalation support. Each ticket tracks the requester, assigned supporter, timestamps, and resolution time.

Service Level Agreements

The SLA Management page defines policies tied to ticket categories. Each SLA tracks response and resolve times, and the system monitors breach status. The dashboard shows held vs. breached SLAs, giving visibility into service level performance.

Knowledge Management

The Articles section functions as a knowledge base. This reduces repeat tickets by enabling self-service and helping supporters resolve issues faster.

Configuration Management

The Supporters section maintains the configuration management database (CMDB) equivalent: support staff roster with names, emails, and phone numbers for contact and assignment.

Service reporting & metrics

The Dashboard sums up ticket data: total tickets, held SLAs, breached SLAs, and resolved times. This gives leadership visibility into service quality and performance.

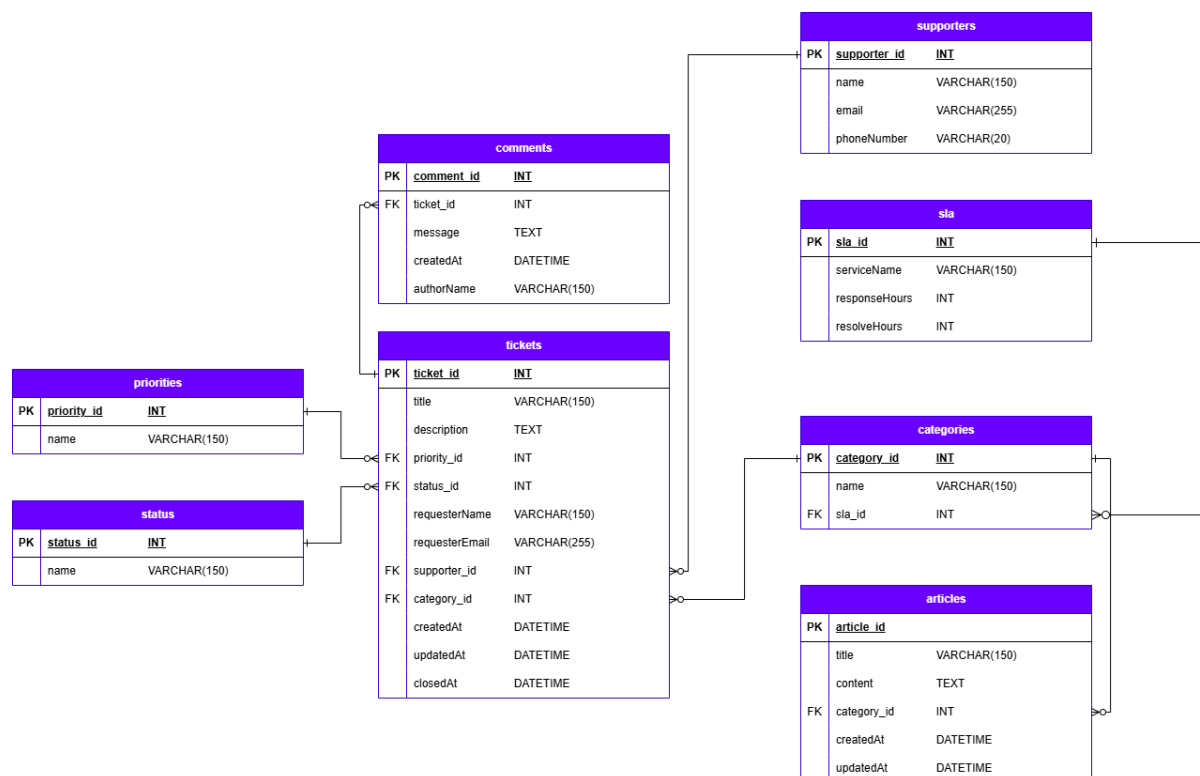
Activity Tracking

Comments on tickets create an audit trail of work done and decisions made, supporting traceability.

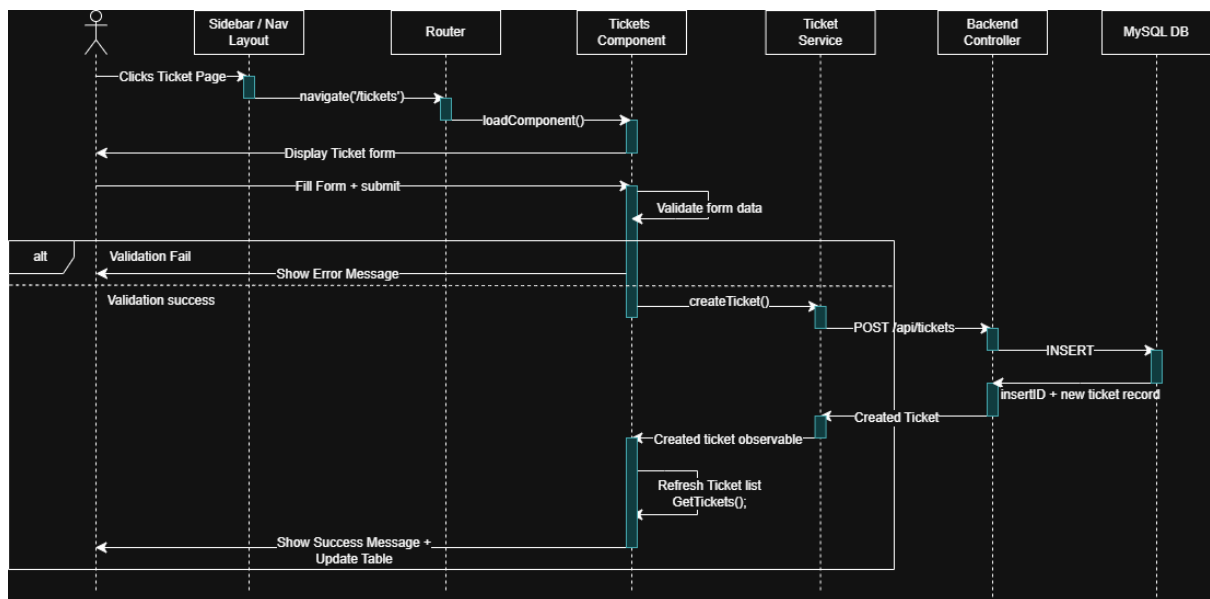
The overall workflow is ITIL-compliant: requests come in as tickets → prioritized and categorized → assigned to supporters → tracked against SLAs → closed and measured → insights fed back to improve the knowledge base. It's a somewhat standard ITIL-service desk pattern.

Diagrams

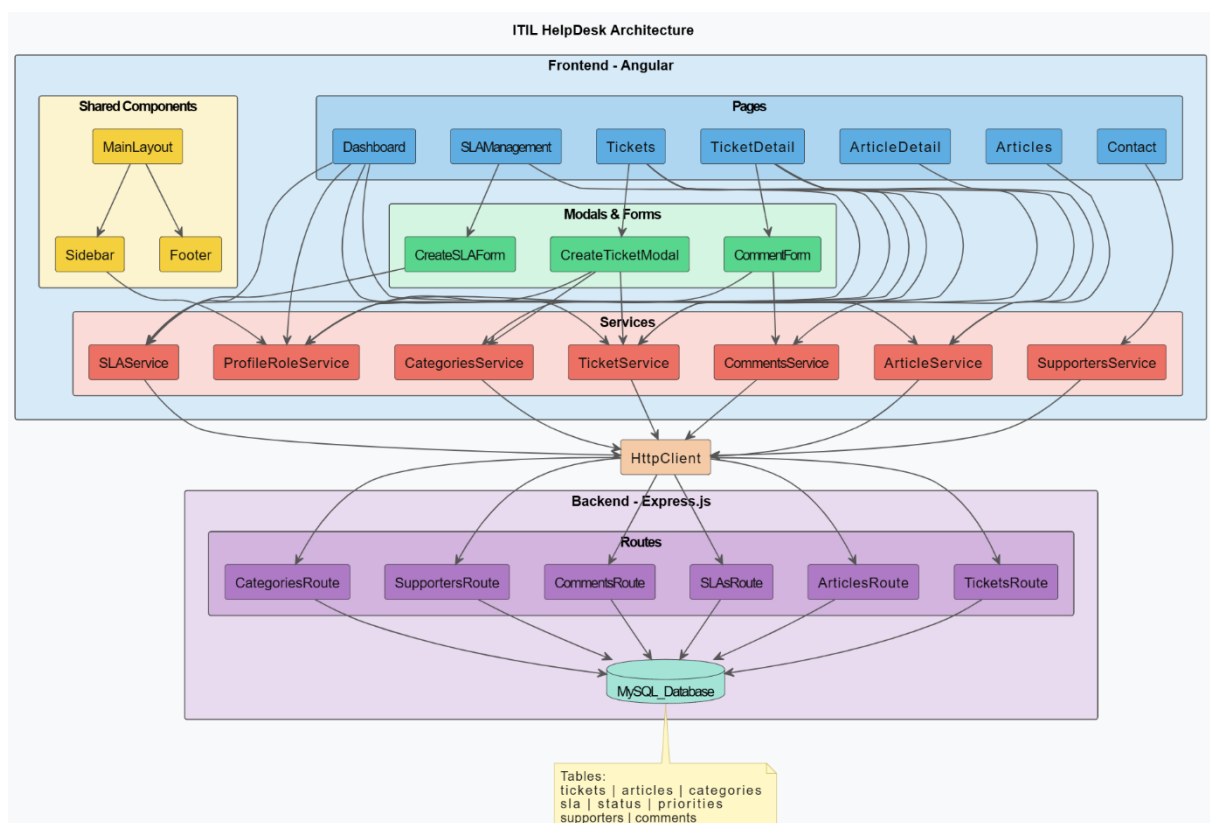
DB-Diagram



Sequence-diagram (Create Ticket)



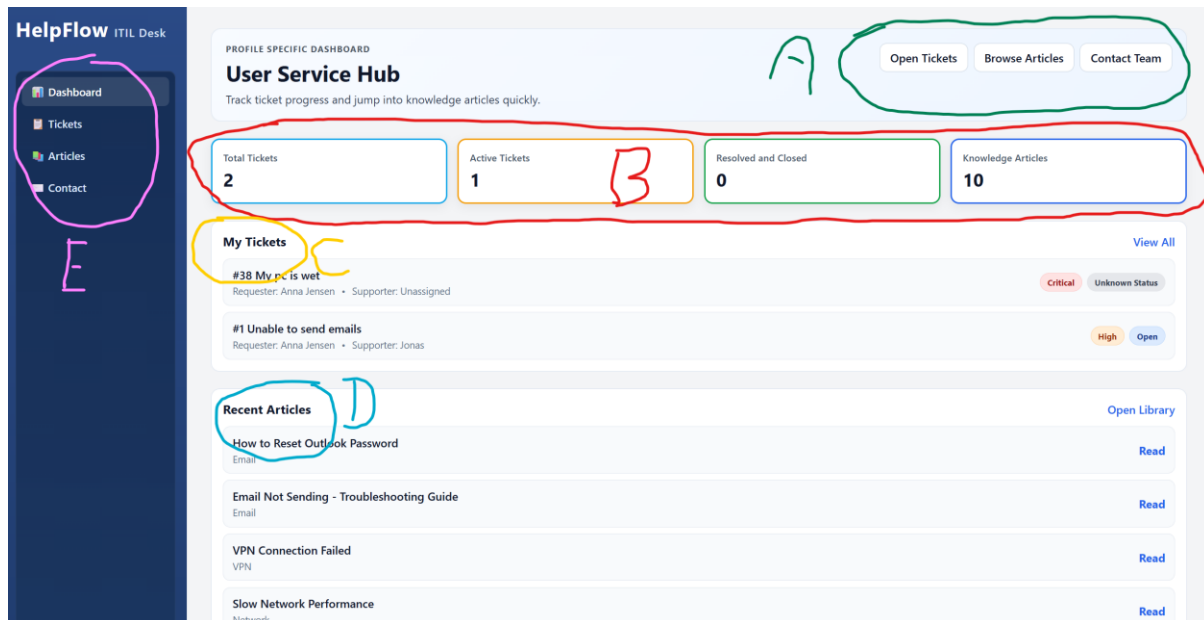
Component-diagram



User Manuals

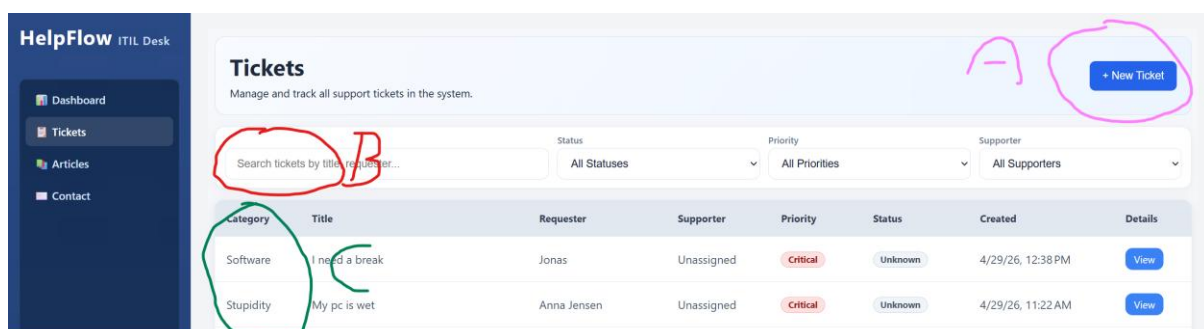
For users

- 1) Login to the system
 - a. After logging in, you will be met with the dashboard of the system
- 2) On the dashboard you can see the following:



- a. Navigation menu
- b. Summation of tickets and articles
- c. Your current tickets
- d. Recently added articles
- e. Navigation sidebar

- 3) You can here access what you need, if you enter tickets, you will be met with:



- a. Button to create a new ticket
- b. Filtering and search options
- c. A list of tickets, with the ability to open each individual ticket for more information.

4) When clicking the new ticket button you need to submit the following form:

- Ticket Title
- Ticket Description
- Category
- Priority
- You can then click the create ticket button to finish your submission

The screenshot shows a 'Create New Ticket' form with the following fields and annotations:

- Title:** 'Test ticket' (annotated with a green 'A')
- Description:** 'Test description' (annotated with a green 'B')
- Requester Name:** 'Anna Jensen'
- Requester Email:** 'anna.jensen@company.local'
- Category:** 'VPN' (annotated with a green 'C')
- Priority:** 'Low' (annotated with a green 'D')
- Buttons:** 'Cancel' and 'Create Ticket' at the bottom right.

5) If you enter the article pages you will be able to access all available articles with quick guides on how to fix common issues.

The screenshot shows the 'Welcome to the Articles Page' with a search bar and three article cards:

- Email - How to Reset Outlook Password:** If a user cannot log in to Outlook, first verify internet connection. Then navigate to the password reset port ... [View Article](#)
- Email - Email Not Sending - Troubleshooting Guide:** Check if Outlook is connected to the mail server. Verify internet access. Review mailbox storage quota. Restart ... [View Article](#)
- VPN - VPN Connection Failed:** Ensure the device has internet connection. Confirm VPN client is updated. Verify username and password. Restart ... [View Article](#)

6) The contact page has the following:

The screenshot shows the 'Contact Us' page with the following sections:

- Our Supporters:** A row of four cards for Jonas, Emma, Martin, and Unassigned. Each card has a 'Contact' button. The Jonas card is circled in green and labeled with a green 'A'.
- Information:** A section with contact details and a 'Contact' button. This section is circled in red and labeled with a red 'B'.

- A card for each supporter. You can contact this supporter directly on the phone or click the contact button for direct email.
- A general information email that doesn't require a specific supporter.

For supporters

- 1) Login to the system
 - a. After logging in, you will be met with the dashboard of the system
- 2) On the dashboard you can see the following:

HelpFlow ITIL Desk

Supporter Command Center

Monitor assignment pressure, SLA health, and urgent incidents in one place.

Assigned Tickets: 12 | Unassigned Tickets: 8 | SLA Held: 6 | SLA Breached: 33 | Active SLA Policies: 6

Escalated and Critical Tickets

- #4 Slow internet connection (Jonas Madsen • Jonas) - Critical, Escalated
- #39 I need a break (Jonas • Unassigned) - Critical, Unknown Status
- #38 My pc is wet (Anna Jensen • Unassigned) - Critical, Unknown Status
- #11 WiFi unavailable in meeting room (Ida Christensen • Jonas) - Critical, Open

My Tickets

- #4 Slow internet connection (Requester: Jonas Madsen • Supporter: Jonas) - Critical, Escalated
- #7 Email spam filtering issue (Requester: Freja Mortensen • Supporter: Jonas) - Low, In Progress

- a. Navigation menu to each of the pages
- b. An overview of
 - i. Assigned tickets, unassigned tickets and SLA overview
 - ii. Escalated and critical tickets
 - iii. Your assigned tickets
 - iv. And recent articles at the bottom
- 3) On the ticket page you can do the following:

HelpFlow ITIL Desk

Tickets

Manage and track all support tickets in the system.

Search tickets by title, requester... | Status: All Statuses | Priority: All Priorities | Supporter: All Supporters

Category	Title	Requester	Supporter	Priority	Status	Created	Details
Software	I need a break	Jonas	Unassigned	Critical	Unknown	4/29/26, 12:38 PM	View
Stupidity	My pc is wet	Anna Jensen	Unassigned	Critical	Unknown	4/29/26, 11:22 AM	View

- a. Button to create a new ticket
- b. Filtering and search options
- c. A list of tickets, with the ability to open each individual ticket for more information.

4) When clicking the new ticket button you need to submit the following form:

- a. Ticket Title
- b. Ticket Description
- c. Category
- d. Priority
- e. You can then click the create ticket button to finish your submission

The screenshot shows a 'Create New Ticket' form with the following fields and values:

- Title:** Test ticket (labeled A)
- Description:** Test description (labeled B)
- Requester Name:** Anna Jensen
- Requester Email:** anna.jensen@company.local
- Category:** VPN (labeled C)
- Priority:** Low (labeled D)

Buttons at the bottom: Cancel, Create Ticket

5) If you access a ticket, you can comment or edit the ticket on the following page:

The screenshot shows the 'TICKET DETAILS' page for ticket #39. The title is '#39 I need a break'. The status is 'Unknown' and the category is 'Software'. The requester is 'Jonas' and the supporter is 'Unassigned'. The priority is 'Critical'. The ticket was created and updated on 'Apr 29, 2026, 12:38:49 PM'. The description is 'I made this ticket so it seems like I am working - but I actually just need a break'. The comments section is empty, with a 'Submit Comment' button. A red 'B' is written next to the comments section.

Buttons at the top right: Edit Ticket, Back to Tickets

- a. Button to edit ticket information or assign a supporter
 - b. Write comments
- 6) If you enter the article pages you will be able to access all available articles with quick guides on how to fix common issues.

Welcome to the Articles Page

Browse knowledge base articles here. Click any article for more detailed information

Filter by category or title...



Email - How to Reset Outlook Password

If a user cannot log in to Outlook, first verify internet connection. Then navigate to the password reset port ...

[View Article](#)



Email - Email Not Sending - Troubleshooting Guide

Check if Outlook is connected to the mail server. Verify internet access. Review mailbox storage quota. Restart ...

[View Article](#)



VPN - VPN Connection Failed

Ensure the device has internet connection. Confirm VPN client is updated. Verify username and password. Restart ...

[View Article](#)

7) If you enter SLA-management, you can do the following:

SLA Management
Create, edit and manage all Service Level Agreement policies.

6 Total SLA Policies

6 Successfully Held SLAs

33 Breached SLAs

SLA ID	Service Name	Response (Hours)	Resolve (Hours)
1	Email Service	2	8
2	Network Service	1	4
3	Printer Service	8	48
4	VPN Service	2	6
5	Software Support	4	24
6	Stupidity service	30	60

- See an overview of the SLA services
- Click the create button to add a new SLA, as shown below.

Create New SLA Policy
New SLA service name will be saved as the category name plus " service".

Category Name:

Response Hours:

Resolve Hours:

8) The contact page has the following:

Contact Us
If you have any questions or need assistance, please feel free to reach out to us. We will attempt to respond as quickly as possible!

Our Supporters

J

Jonas
Email: jonas@email.com
Phone: 12345678

E

Emma
Email: emma@email.com
Phone: 12345678

M

Martin
Email: martin@email.com
Phone: 12345678

U

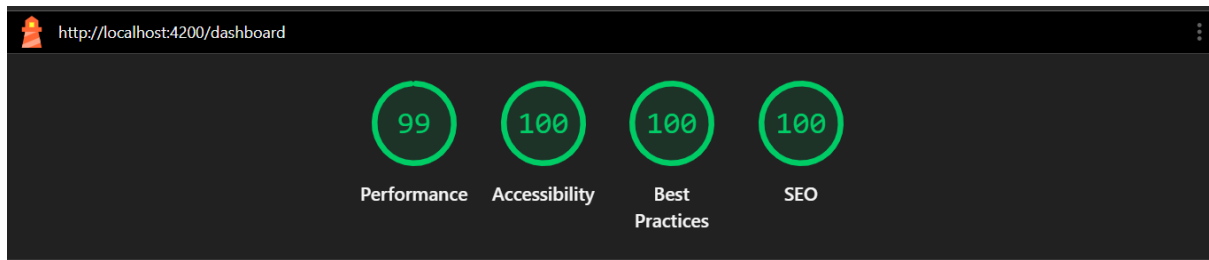
Unassigned
Email: unassigned@email.com
Phone: 12345678

Information
For general inquiries, please email us at: support@helpflow.com
For urgent issues, please call us at: 123-456-7890
We are happy to assist you with any questions or concerns you may have. (>OwO)>

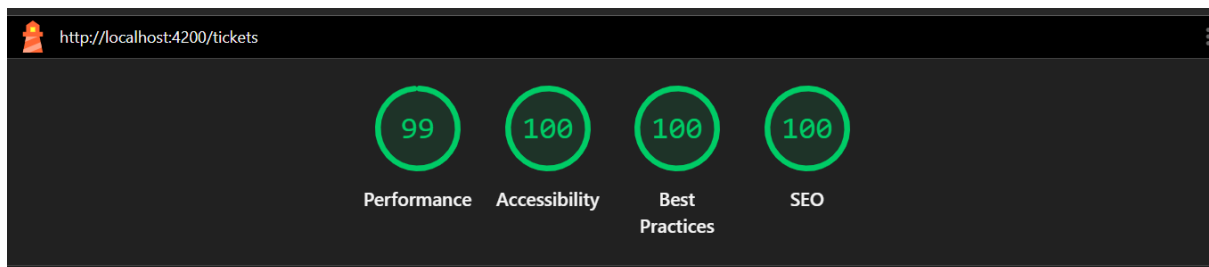
- A card for each supporter. You can contact this supporter directly on the phone or click the contact button for direct email.
- A general information email that doesn't require a specific supporter.

Lighthouse test

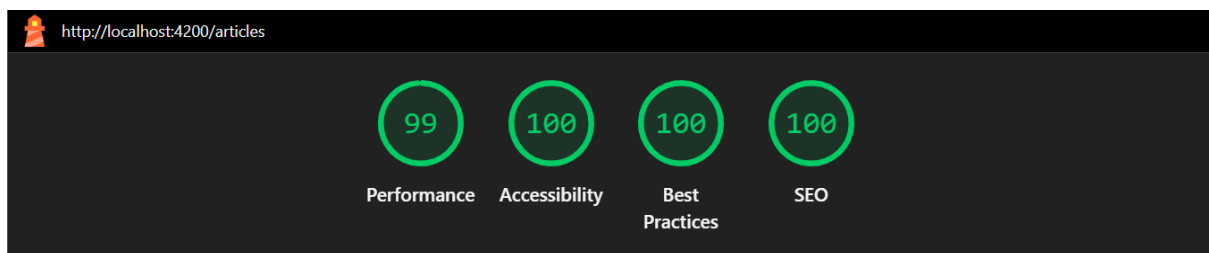
Dashboard



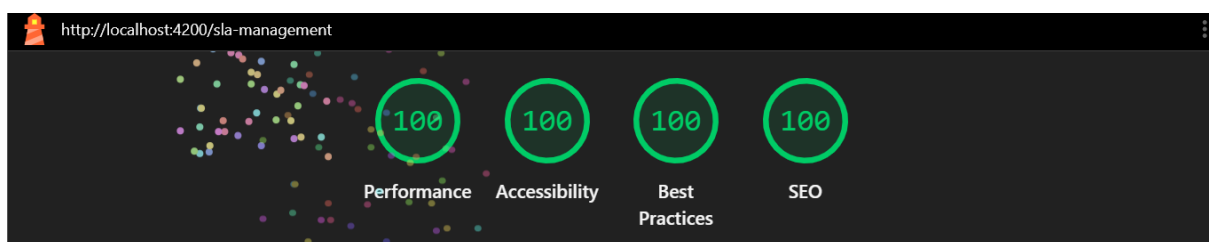
Tickets



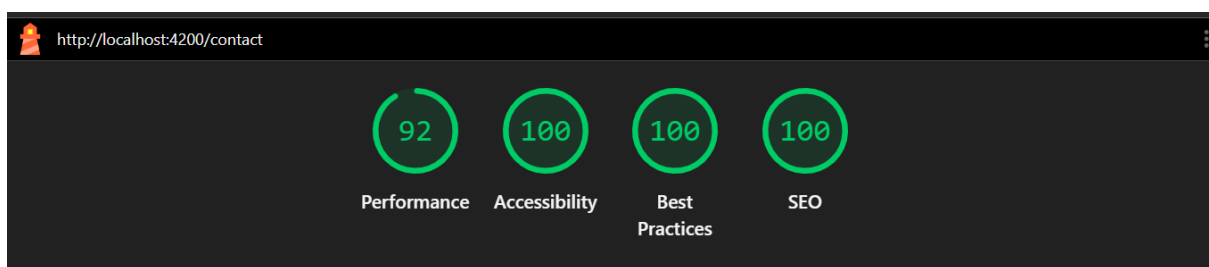
Articles



SLA-management



Contact



There is a layout shift issue with my div.information-section that I haven't been able to figure out entirely.

Retrospective

Some of the things I would have done differently if I had to develop this project again are the following:

- Use of global styles

I would have used more global styles to design my website. I have a lot of duplicate CSS defining headlines, meta-data and the like. This could've easily been designed in a global styles.css file and saved me space in the individual CSS documents. This could contribute to maintainability and refactorability.

- Spend more time on mockups and design ideas

This could've been helpful for the overall development of the system, as I would have a clearer picture of how I wanted the site to look. I feel like I pigeonholed myself too early into a set design, where I might have been able to be more creative and found a more interesting layout.

- Done more research into other/existing service desks

This could have given me a better point of view regarding the project and the product I had to develop. There is no point in reinventing the wheel and could most likely have saved me some time on brainstorming.

- Multithreading

Multithreading could be useful for large report calculations if a report page was added later in development. It could be used to calculate trends, SLA simulations over many tickets and use worker thread that would not block API calls. I am currently just using ASYNC for my request handlers which currently works perfectly.

Conclusion

I feel like I managed to accomplish what I wanted with this project. I knew it was a challenge to pick both a new framework (Angular) and a new backend (express) that I had no experience with. I also used typescript for the first time. If I had to do it again, I would most likely either stick with this setup or use REACT with Vite instead. I had a lot of trouble getting started with Angular since the component creation differs a lot from what I am used to.

After working with it for a while, it became more manageable and I feel like I am comfortable with it now. All in all, I feel like the project turned out well and I am satisfied with what I have accomplished.

Installation / API / Dependency setup

Can be read in the README.md on the github page <https://github.com/MHLoyche/ITILHelpDesk>